
OpenISAC

Extensión ZMQ Software-in-the-Loop

Canal ISAC simulado con interfaz TUI interactiva

Documentación técnica: arquitectura, modelo de canal, cambios al código y manual de uso

Resumen ejecutivo

Este documento describe la extensión *software-in-the-loop* (SIL) desarrollada sobre el proyecto de código abierto OpenISAC (disponible en GitHub). La extensión añade un modo de simulación via ZeroMQ (- - zmq) que permite ejecutar la cadena completa TX → Canal → RX de procesamiento ISAC sin ningún hardware de radio (USRP), sustituyendo la interfaz SDR por dos herramientas de simulación:

- **ChannelSimulator_void**: canal transparente (pasa la señal sin modificar), útil como referencia para verificar que no exista actividad cuando no hay objetos.
- **ChannelSimulator**: canal radar monostático completo con retardo de rango, desplazamiento Doppler, micro-Doppler, ecuación del radar y ruido AWGN, controlable en tiempo real mediante una interfaz TUI (ncurses).

El documento incluye la formulación matemática rigurosa del modelo de canal, las referencias exactas al código que implementa cada efecto, y las instrucciones paso a paso para ejecutar el sistema tanto en modo hardware (USRP) como en modo simulado.

Proyecto base	OpenISAC (fork) — https://github.com/OpenISAC
Lenguaje	C++17
Build system	CMake ≥ 3.18
Plataforma	Linux (Ubuntu 22.04+), kernel ≥ 5.15
Dependencias SIL	ZeroMQ ≥ 4.3 , cppzmq, ncurses, FFTW3
Fecha	Mayo 2026

Documento de referencia para publicación en congreso sobre ISAC. Generado con
XeLaTeX — fork_OpenISAC.

0 Contents

1	Introducción y motivación	3
2	Arquitectura del sistema completo	3
2.1	Modos de operación	3
2.2	Pipeline de señal en modo ZMQ SIL	4
2.3	Diagramas de flujo por modo de ejecución	4
2.3.1	Modo 1 — Hardware real (USRP)	4
2.3.2	Modo 2 — Canal void (referencia nula, sin USRP)	5
2.3.3	Modo 3 — Canal simulado con efectos físicos (sin USRP)	5
2.4	Flujo de datos detallado	5
3	Modelo de canal implementado	6
3.1	Retardo de rango	6
3.2	Desplazamiento Doppler	7
3.3	Micro-Doppler	8
3.4	Amplitud del eco — ecuación del radar normalizada	9
3.5	Diagrama de radiación de antena directiva	10
3.6	Ruido AWGN	11
3.7	Tabla de parámetros del canal	11
4	Canal transparente: ChannelSimulator_void	12
5	Cambios al código original de OpenISAC	13
5.1	Archivos nuevos	13
5.2	Archivos modificados	13
5.2.1	src/OFDMModulator.cpp	13
5.2.2	src/SensingChannel.cpp / include/SensingChannel.hpp	16
5.2.3	CMakeLists.txt	17
5.3	Correcciones de bugs críticos detectados	18
5.3.1	Bug 1: Bloqueo infinito en PairedFrameQueue (modo ZMQ)	18
5.3.2	Bug 2: Fuga de memoria de 14,7 GB en _zmq_sensing_rx_proc	18
5.3.3	Bug 3: SIGTERM sin manejador registrado	19
6	Manual de instalación	19
6.1	Dependencias del sistema	19
6.2	Compilación	19
6.3	Configuración YAML	20
7	Manual de uso paso a paso	20
7.1	Modo 1: Hardware real (USRP)	20
7.1.1	Terminal 1 — Configuración del sistema (una vez por sesión)	20
7.1.2	Terminal 2 — OFDMModulator (sin flag -zmq)	21
7.1.3	Terminal 3 — Visualización	21
7.2	Modo 2: Canal transparente (ChannelSimulator_void)	21
7.2.1	Terminal 1 — ChannelSimulator_void	21
7.2.2	Terminal 2 — OFDMModulator en modo ZMQ	21
7.2.3	Terminal 3 — Visualización	22
7.2.4	Parada limpia	22
7.3	Modo 3: Canal simulado con efectos físicos (ChannelSimulator)	22
7.3.1	Terminal 1 — ChannelSimulator (con TUI)	22

7.3.2	Terminal 2 — OFDMModulator en modo ZMQ	22
7.3.3	Terminal 3 — Visualización	22
7.3.4	Uso de la TUI ncurses	22
7.3.5	Experimentos sugeridos	23
7.4	Control remoto por UDP (opcional)	24
8	Rendimiento y optimizaciones en tiempo real	24
8.1	Tabla de ruido precalculada	25
8.2	Fasor Doppler incremental	25
8.3	Hilo de señal con prioridad RT	25
9	Resolución de problemas	26
10	Resumen de todos los cambios frente al repositorio original	27

1 Introducción y motivación

OpenISAC ¹ es una implementación de código abierto en C++ de un sistema ISAC (*Integrated Sensing and Communications*) basado en OFDM (*Orthogonal Frequency-Division Multiplexing*), diseñado para ejecutarse sobre hardware SDR (*Software-Defined Radio*) de la familia USRP de Ettus Research / National Instruments.

El proyecto original asume disponibilidad de hardware físico para *cualquier* prueba del sistema, lo que impone restricciones importantes en entornos de desarrollo, validación de algoritmos y docencia:

- La cadena de procesamiento ISAC completa no puede ejecutarse sin un USRP conectado.
- No existe mecanismo para controlar los parámetros del canal (distancia, velocidad, SNR) de forma reproducible entre experimentos.
- La validación de los algoritmos de procesamiento radar (mapa Rango-Doppler, CFAR) queda condicionada a la disponibilidad del entorno de medidas.

Esta extensión, denominada **ZMQ SIL** (*Software-in-the-Loop*), resuelve estos problemas añadiendo:

1. Un modo de transmisión ZMQ (--zmq) que sustituye la interfaz USRP por sockets ZeroMQ PUSH/PULL, sin modificar la cadena de procesamiento.
2. Un canal simulado transparente (ChannelSimulator_void) para obtener una referencia sin efectos de canal.
3. Un canal simulado físico (ChannelSimulator) que implementa un modelo radar monostático parametrizable en tiempo real.

2 Arquitectura del sistema completo

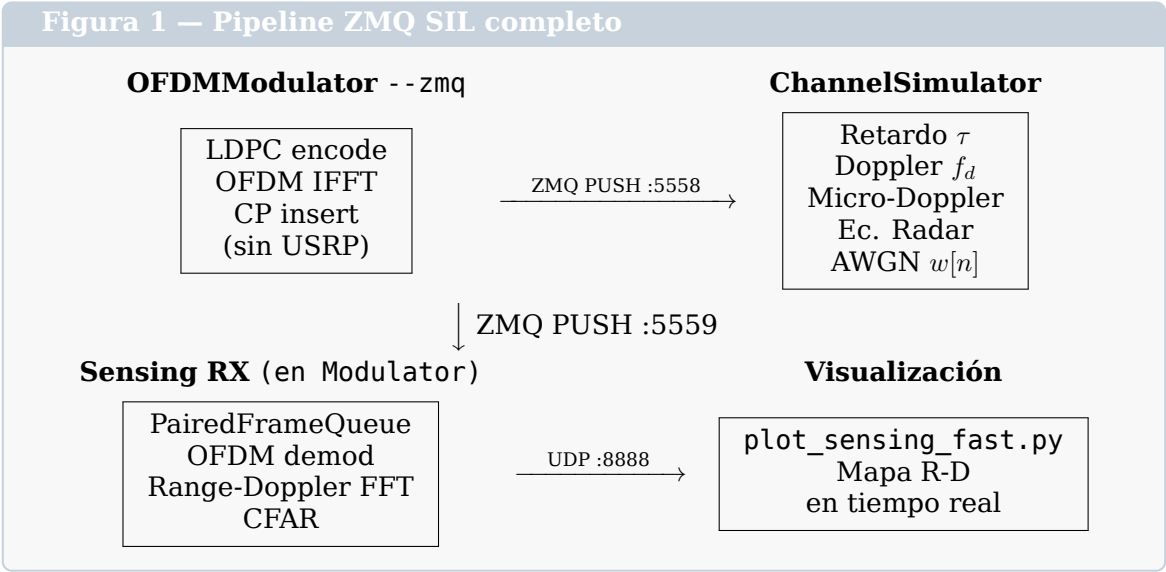
2.1 Modos de operación

El sistema puede operar en dos modos mutuamente excluyentes:

Modo	Interfaz TX/RX	Casos de uso
Hardware (normal)	USRP via UHD	Experimentos OTA, medidas reales
ZMQ SIL (--zmq)	ZeroMQ sockets	Desarrollo, validación, docencia

¹<https://github.com/OpenISAC>

2.2 Pipeline de señal en modo ZMQ SIL

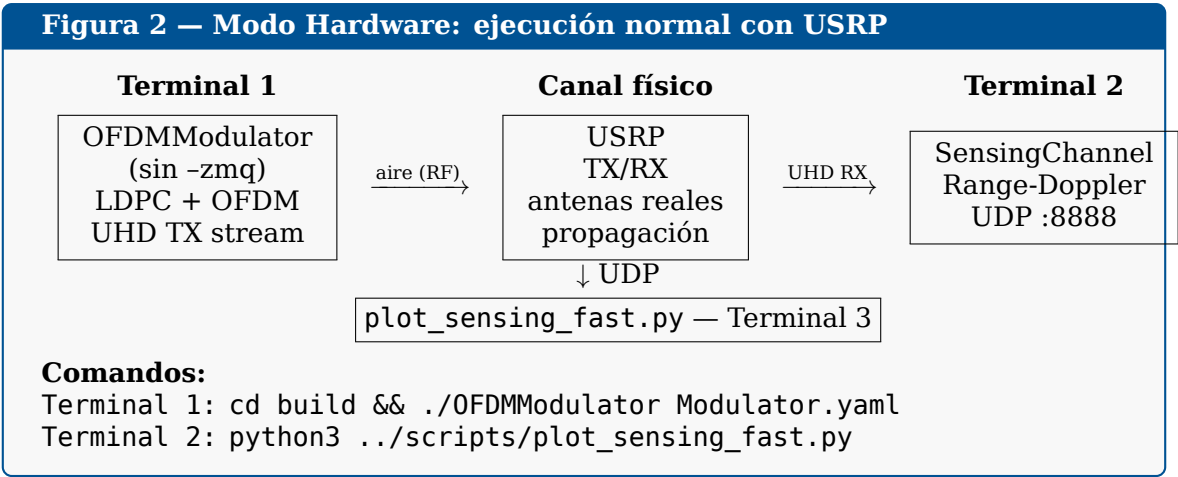


En modo hardware, la señal **se transmite por el aire** entre antenas TX y RX del USRP. En modo ZMQ SIL, la señal se transfiere por **memoria local** a través de sockets ZeroMQ, y el canal físico es emulado por software.

2.3 Diagramas de flujo por modo de ejecución

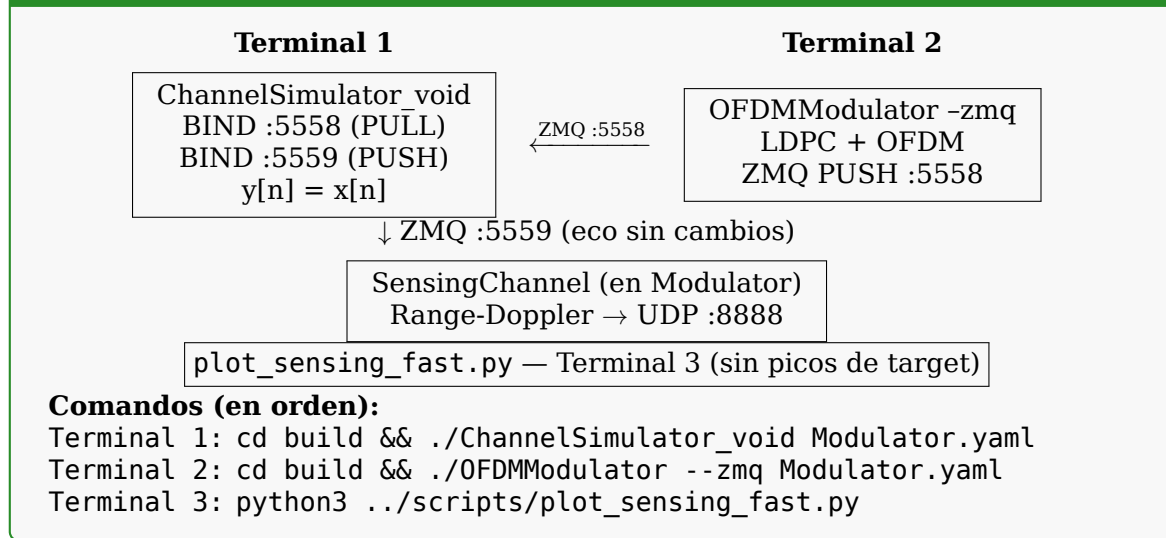
Los tres modos de ejecución posibles se ilustran a continuación de forma independiente.

2.3.1 Modo 1 — Hardware real (USRP)



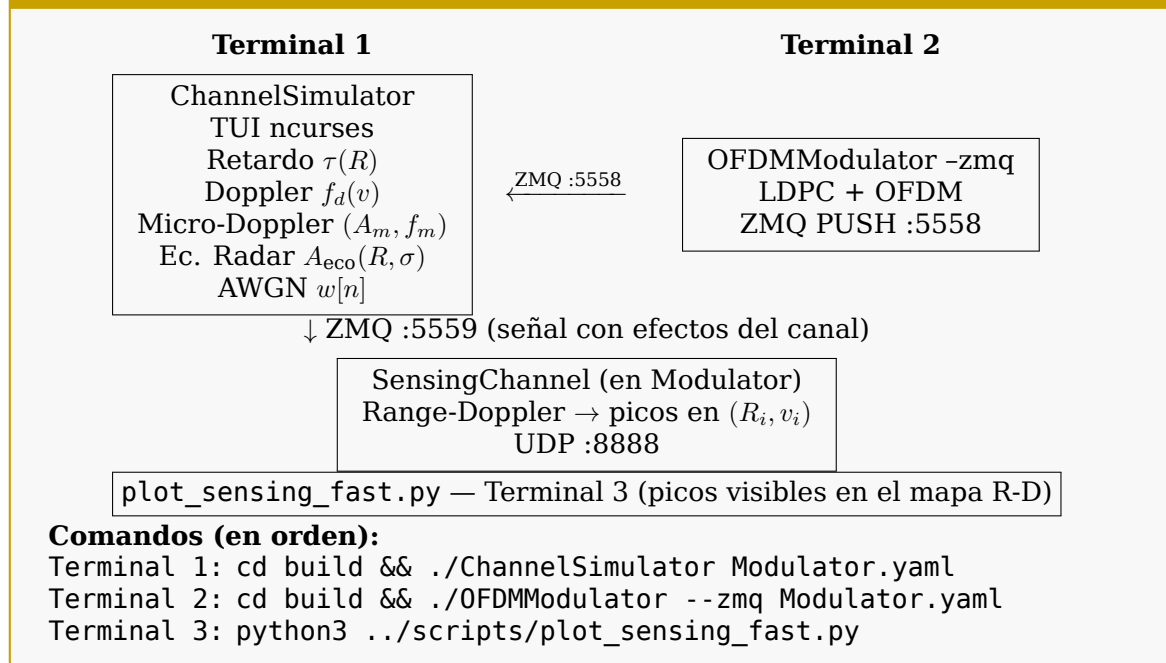
2.3.2 Modo 2 — Canal void (referencia nula, sin USRP)

Figura 3 — Modo Canal Void: sin efectos de canal



2.3.3 Modo 3 — Canal simulado con efectos físicos (sin USRP)

Figura 4 — Modo Canal Simulado: efectos físicos controlables



2.4 Flujo de datos detallado

1. **OFDMModulator (hilo_modulation_proc):** codifica los datos de usuario con LDPC, modula con OFDM (IFFT + CP) y deposita cada frame de $N_{\text{sym}} \times (N_{\text{FFT}} + N_{\text{CP}})$ muestras complejas en el buffer circular (`_circular_buffer`).
2. **OFDMModulator (hilo_tx_proc)** — modo ZMQ: extrae el frame del buffer circular y lo envía via ZMQ PUSH al puerto 5558. Simultáneamente, encola los símbolos en frecuencia en PairedFrameQueue (lado TX) para el procesamiento radar posterior.

3. **ChannelSimulator**: recibe el frame por ZMQ PULL (puerto 5558), aplica los efectos del canal (véase Sección 3) y envía el resultado por ZMQ PUSH (puerto 5559).
4. **OFDMModulator (hilo _zmq_sensing_rx_proc)**: recibe el frame procesado por ZMQ PULL (puerto 5559) y lo inyecta en PairedFrameQueue (lado RX), haciendo corresponder cada frame RX con su frame TX por número de secuencia.
5. **OFDMModulator (hilo _sensing_loop)**: extrae pares (TX, RX) de PairedFrameQueue, aplica demodulación OFDM al lado RX y calcula el mapa Rango-Doppler mediante FFT 2D. El resultado se envía via UDP al puerto 8888 para visualización con `plot_sensing_fast.py`.

3 Modelo de canal implementado

El ChannelSimulator implementa un modelo físico de canal radar monostático para $N_T \leq 3$ objetivos (*targets*) independientes. El modelo es continuo en tiempo discreto: se aplica muestra a muestra sobre el flujo I/Q recibido.

La señal de salida del canal es la **superposición de los ecos de todos los targets más ruido AWGN**:

Señal compuesta de salida del canal

$$y[n] = \sum_{i=1}^{N_T} A_{\text{eco},i} \cdot x[n - d_i] \cdot p_i[n] + w[n] \quad (1)$$

donde $x[n]$ es la señal transmitida, $A_{\text{eco},i}$ la amplitud del eco del target i , d_i el retardo en muestras, $p_i[n]$ el fador de Doppler (con micro-Doppler opcional) y $w[n]$ el ruido aditivo.

3.1 Retardo de rango

El eco del target i a distancia R_i se recibe con un retardo de ida y vuelta:

$$\tau_i = \frac{2R_i}{c} \quad [\text{s}] \quad (2)$$

donde $c = 2,998 \times 10^8$ m/s es la velocidad de la luz. El retardo en muestras enteras es:

$$d_i = \left\lfloor \frac{2R_i}{c} \cdot f_s + 0,5 \right\rfloor \quad [\text{muestras}] \quad (3)$$

con f_s la frecuencia de muestreo del sistema (parámetro `sample_rate` en el YAML).

El retardo se implementa mediante un **buffer circular** de profundidad $d_{\text{max}} = \lceil 2 \times 20000/c \times f_s \rceil + N_{\text{frame}} + 1$ (suficiente para alojar el eco más lejano de la TUI: $R_{\text{max}} = 10$ km, round-trip = 20 km):

```
1 // Profundidad del buffer circular: round-trip de 20 km + margen
2 const size_t max_delay_samps =
3     static_cast<size_t>(std::ceil(2.0f * 20000.0f / C_LIGHT * fs))
4     + buffer_size + 1u;
5 std::vector<std::complex<float>> ring(max_delay_samps, {0.f, 0.f});
6
```



```

7 // ...
8 // Escritura en el buffer circular con el frame entrante
9 for (ssize_t i = 0; i < n; ++i)
10     ring[(ring_write + static_cast<size_t>(i)) % max_delay_samps] = in_buf[i];

```

Listing 1: Implementación del buffer circular de retardo — ChannelSimulator.cpp, líneas 68-72 y 113-115

```

1 // d_i = round(2*R_i / c * f_s) [ecuacion eq:delay]
2 const size_t delay_samps =
3     static_cast<size_t>(std::round(2.0f * dist / C_LIGHT * fs));
4
5 // ...
6 // Lectura del buffer con el retardo correspondiente al target i
7 const size_t src =
8     (ring_write + static_cast<size_t>(i) + max_delay_samps - delay_samps)
9     % max_delay_samps;

```

Listing 2: Cálculo del retardo y lectura desde el buffer — ChannelSimulator.cpp, líneas 129-130 y 156-159

Resolución en rango

La resolución en rango del procesado ISAC es $\Delta R = c/(2B)$, donde B es el ancho de banda de la señal OFDM ($B \approx f_s$ para subportadoras densas). Con $f_s = 50$ MHz: $\Delta R = 3$ m/muestra (muestras de tiempo) vs. $\Delta R_{\text{ISAC}} = c/2B \approx 3$ m (bin Range-FFT). El retardo del canal está cuantificado a resolución de 1 muestra, coherente con la resolución del procesador ISAC.

3.2 Desplazamiento Doppler

Un target que se mueve con velocidad radial v_i (positivo = aproximándose) introduce un desplazamiento Doppler en la señal recibida:

$$f_{d,i} = \frac{2v_i f_c}{c} \quad [\text{Hz}] \quad (4)$$

donde f_c es la frecuencia portadora (parámetro `center_freq` en el YAML).

El desplazamiento Doppler se implementa como una **multiplicación por un fasor complejo giratorio de amplitud unitaria**. El fasor se actualiza de forma incremental entre muestras consecutivas, evitando el cálculo de funciones trigonométricas en el bucle interno (RT critical path):

$$\Delta\phi_i = \frac{2\pi f_{d,i}}{f_s} \quad (5)$$

$$p_i[n+1] = p_i[n] \cdot e^{j\Delta\phi_i} = p_i[n] \cdot (\cos \Delta\phi_i + j \sin \Delta\phi_i) \quad (6)$$

La multiplicación del fasor es una operación de **2 multiplicaciones + 2 sumas reales** (rotación compleja), frente a las ≈ 15 ns de $\sin/\cos$ en hardware x86. El fasor se renormaliza a módulo unidad cada 256 frames para compensar la deriva acumulada de punto flotante.

```

1 // Delta de fase Doppler por muestra: d_phi = 2*pi * f_d / fs [ec. eq:doppler_phi]
2 // f_d = 2*vel*fc/c [ec. eq:doppler_freq]
3 const float d_phi = 2.0f * PI * (2.0f * vel * fc / C_LIGHT) / fs;
4 const std::complex<float> d_rot{std::cos(d_phi), std::sin(d_phi)};

```

Listing 3: Inicialización del fasor Doppler — ChannelSimulator.cpp, líneas 146-147

```

1 // Rotacion incremental: p[n+1] = p[n] * d_rot [ec. eq:doppler_phasor]
2 // Implementada como multiplicacion de complejos (sin llamar a sin/cos por muestra)
3 dop_phasor[ti] = {
4     dop_phasor[ti].real() * d_rot.real() - dop_phasor[ti].imag() * d_rot.imag(),
5     dop_phasor[ti].real() * d_rot.imag() + dop_phasor[ti].imag() * d_rot.real()
6 };

```

Listing 4: Avance incremental del fasor — ChannelSimulator.cpp, líneas 168-172

```

1 // Renormalizar phasors cada 256 frames para evitar deriva de punto flotante.
2 // La multiplicacion acumulada de floats introduce error 0(eps * n_samples).
3 if (++renorm_ctr >= 256) {
4     renorm_ctr = 0;
5     for (int ti = 0; ti < 3; ++ti) {
6         const float mag = std::abs(dop_phasor[ti]);
7         if (mag > 0.0f) dop_phasor[ti] /= mag;
8     }
9 }

```

Listing 5: Renormalización del fasor cada 256 frames — ChannelSimulator.cpp, líneas 184-191

3.3 Micro-Doppler

El micro-Doppler modela la modulación adicional en fase introducida por la vibración o movimiento periódico de partes del target (p.ej. hélices, extremidades). Se implementa como una modulación en fase sinusoidal superpuesta al fasor Doppler principal:

$$\phi_m(n) = A_m \sin\left(\frac{2\pi f_m n}{f_s}\right) \quad (7)$$

El fasor combinado (Doppler + micro-Doppler) es:

$$p_i[n] = e^{j(\phi_{d,i}(n) + A_m \sin(2\pi f_m n / f_s))} \quad (8)$$

donde A_m [rad] es la amplitud de la desviación de fase y f_m [Hz] la frecuencia de vibración. Este efecto produce *sidebands* simétricas en el espectro Doppler separadas $\pm k f_m$ de la frecuencia Doppler principal, característica de objetos con partes en movimiento.

La implementación avanza el acumulador de micro-Doppler sample a sample mediante las fórmulas de adición trigonométrica (sin llamadas a sin/cos en el bucle):

```

1 // md_phase[ti] acumula la fase del oscilador de micro-Doppler entre frames
2 // md_step = 2*pi*f_m / fs (incremento de fase por muestra)
3 const float md_step = 2.0f * PI * md_freq / fs;
4 float md_sin = std::sin(md_phase[ti]); // sin(phi_m[n])
5 float md_cos = std::cos(md_phase[ti]); // cos(phi_m[n])
6 const float md_sin_s = std::sin(md_step); // sin(Delta_phi_m)
7 const float md_cos_s = std::cos(md_step); // cos(Delta_phi_m)

```

Listing 6: Inicialización del acumulador micro-Doppler — ChannelSimulator.cpp, líneas 149-154

```

1 std::complex<float> phasor = dop_phasor[ti];
2 if (md_amp > 0.0f) {
3     // phi_micro = A_m * sin(phi_m[n])           [ec. eq:microdoppler_phase]
4     // phasor_total = phasor_doppler * exp(j * phi_micro) [ec. eq:phasor_combined]
5     const float md_mod = md_amp * md_sin;
6     phasor *= std::complex<float>{std::cos(md_mod), std::sin(md_mod)};
7 }
8 out_buf[i] += echo_amp * ring[src] * phasor; // suma contribucion del target i
9
10 // Avance del oscilador de micro-Doppler usando suma de angulos (sin trig por muestra):
11 // sin(phi + Delta) = sin(phi)*cos(Delta) + cos(phi)*sin(Delta)
12 // cos(phi + Delta) = cos(phi)*cos(Delta) - sin(phi)*sin(Delta)
13 if (md_amp > 0.0f) {
14     const float ns = md_sin * md_cos_s + md_cos * md_sin_s;
15     const float nc = md_cos * md_cos_s - md_sin * md_sin_s;
16     md_sin = ns; md_cos = nc;
17 }

```

Listing 7: Aplicación del micro-Doppler al fasor — ChannelSimulator.cpp, líneas 161-178

3.4 Amplitud del eco — ecuación del radar normalizada

La potencia de la señal recibida en un radar monostático viene dada por la **ecuación del radar**:

$$P_r = \frac{P_t G_t G_r \lambda^2 \sigma_{\text{RCS}}}{(4\pi)^3 R^4} \quad (9)$$

donde P_t es la potencia transmitida, G_t y G_r las ganancias de antena TX y RX, $\lambda = c/f_c$ la longitud de onda, σ_{RCS} la sección eficaz radar (*Radar Cross Section*) y R la distancia al target.

En el dominio de amplitud de señal ($A = \sqrt{P}$), la amplitud del eco recibido escala como R^{-2} . Para la simulación se usa una forma normalizada de la ecuación:

Amplitud del eco — forma implementada

$$A_{\text{eco},i} = K_{\text{sim}} \cdot G(\theta_i) \cdot \lambda \cdot \frac{\sqrt{\sigma_{\text{RCS},i}}}{R_i^2} \cdot 10^{P_{\text{TX}}/20} \cdot 10^{G_{\text{RX}}/20} \quad (10)$$

donde $K_{\text{sim}} = 120$ es la constante de normalización calibrada para obtener $A_{\text{eco}} \approx 0.01$ a las condiciones de referencia: $R = 25$ m, $\sigma_{\text{RCS}} = 1$ m², antena omni ($G = 1$), $f_c = 5,8$ GHz, $P_{\text{TX}} = G_{\text{RX}} = 0$ dB.

Verificación numérica de $K_{\text{sim}} = 120$:

A las condiciones de referencia, $\lambda = c/f_c = 3 \times 10^8$ m/s / $5,8 \times 10^9$ Hz $\approx 0,0517$ m. Con $G(\theta) = 1$ (omni), $\sqrt{\sigma_{\text{RCS}}} = 1$ m, $R^2 = 25^2 = 625$ m² y $P_{\text{TX}} = G_{\text{RX}} = 0$ dB ($10^{0/20} = 1$):

$$A_{\text{eco}} = K_{\text{sim}} \cdot \frac{G \cdot \lambda \cdot \sqrt{\sigma_{\text{RCS}}}}{R^2} = 120 \cdot \frac{1 \cdot 0,0517 \cdot 1}{625} = \frac{6,204}{625} \approx 0,00993 \approx 0,01 \quad (11)$$

El valor $K_{\text{sim}} = 120$ (KSIM_NORM en ChannelSimulator.cpp) se elige de modo que la amplitud del eco sea del orden de 10^{-2} , lo que sitúa la señal útil entre el piso de ruido ($\sigma_n \approx 10^{-3}$ con $N_{\text{dB}} = -60$ dBP) y la saturación de la escala unitaria, proporcionando un margen dinámico operativo de ≈ 20 dB.

```

1 // Ganancia TX y RX en amplitud: 10^(dB/20)
2 const float tx_scale = std::pow(10.0f, tx_power_db / 20.0f);
3 const float rx_scale = std::pow(10.0f, rx_gain_db / 20.0f);
4
5 // Ganancia de antena maxima (antena directiva Gaussiana)
6 // G_max = 16*ln2 / theta_3dB^2 (antena de haz Gaussiano)
7 // Para antena omni: G = 1.
8 const float G = (ant_type == 0 || ant_bw_deg <= 0.0f) ? 1.0f : [&] {
9     const float bw = ant_bw_deg * PIf / 180.0f; // radianes
10    return 16.0f * 0.6931472f / (bw * bw); // G_max
11 }();
12
13 // ...
14 // Patron de radiacion Gaussiano para la direccion del target
15 float beam_factor = 1.0f;
16 if (ant_type == 1 && ant_bw_deg > 0.0f) {
17     const float theta_half = (ant_bw_deg * 0.5f) * PIf / 180.0f;
18     const float theta = std::abs(angle_d) * PIf / 180.0f;
19     // G(theta) = exp(-ln2 * (theta / theta_half)^2) [ec. eq:beam_pattern]
20     beam_factor = std::exp(-0.6931472f * (theta / theta_half) * (theta / theta_half));
21 }
22
23 // Amplitud del eco: A_eco = Ksim * G * G(theta) * lambda * sqrt(RCS) / R^2
24 // * tx_scale * rx_scale [ec. eq:echo_amp]
25 // lambda = C_LIGHT / fc (calculado al inicio de signal_loop)
26 const float echo_amp = KSIM_NORM * G * beam_factor * lambda * std::sqrt(rcs)
27 / (dist * dist) * tx_scale * rx_scale;

```

Listing 8: Cálculo de la amplitud del eco — ChannelSimulator.cpp, líneas 105-143

3.5 Diagrama de radiación de antena directiva

Para el modo de antena directiva (`antenna_type = 1`), el diagrama de radiación se aproxima con una forma Gaussiana, que es una buena aproximación al lóbulo principal de antenas de apertura y de bocina:

$$G(\theta) = G_{\max} \cdot \exp\left(-\ln 2 \cdot \left(\frac{\theta}{\theta_{3\text{dB}}/2}\right)^2\right) \quad (12)$$

donde θ es el ángulo off-boresight del target, $\theta_{3\text{dB}}$ el ancho de haz a -3 dB (`antenna_beamwidth_deg`) y $G_{\max}$ la ganancia de pico:

$$G_{\max} = \frac{16 \ln 2}{\theta_{3\text{dB}}^2} \quad (\theta_{3\text{dB}} \text{ en radianes}) \quad (13)$$

Esta expresión se obtiene imponiendo que $G(\theta_{3\text{dB}}/2) = G_{\max}/2$ (definición de los 3 dB) en la forma Gaussiana y que la integral del patrón sobre el espacio sólido total sea la unidad (conservación de energía en la aproximación de campo lejano).

Para antena omnidireccional

Cuando `antenna_type = 0` (Omni), se usa $G(\theta) = 1$ para todo θ , lo que equivale a $G_{\max} = 1$ y $\theta_{3\text{dB}} \rightarrow \infty$.

3.6 Ruido AWGN

Se añade ruido gaussiano blanco aditivo complejo (*Additive White Gaussian Noise*) con densidad espectral de potencia uniforme. El parámetro de control es la **potencia del ruido** N_{dB} expresada en dB con referencia a potencia unidad:

$$\sigma_n^2 = 10^{N_{\text{dB}}/10}, \quad \sigma_n = 10^{N_{\text{dB}}/20} \quad (14)$$

$$w[n] \sim \mathcal{CN}(0, \sigma_n^2), \quad w[n] = \sigma_n \cdot \mathcal{N}(0, 1) + j\sigma_n \cdot \mathcal{N}(0, 1) \quad (15)$$

Nota sobre el modelo de ruido

El parámetro `noise_power_db` actúa como **piso de ruido absoluto**: es independiente de la ganancia de recepción `rx_gain_db`. Esto modela un sistema donde el ruido entra en la antena (ruido de antena + temperatura ambiente), mientras que `rx_gain_db` actúa como un amplificador que mejora la SNR del eco pero no afecta al piso de ruido (figura de ruido nula del amplificador). Para simular una figura de ruido finita, incrementar `noise_power_db` y `rx_gain_db` simultáneamente en la misma cantidad.

La generación de ruido usa una **tabla precalculada** de $2^{17} = 131\,072$ muestras Gaussianas (512 KB, cabe en caché L3) para evitar la latencia del generador `std::normal_distribution` en el bucle RT (≈ 15 ns/muestra vs. ≈ 2 ns con tabla):

```

1 // Tabla de ruido precalculada (una vez al inicio, semilla fija = 42 para
  reproducibilidad)
2 std::vector<float> noise_tbl(NOISE_TBL_SZ); // NOISE_TBL_SZ = 2^17 = 131072
3 {
4     std::mt19937 rng0{42u};
5     std::normal_distribution<float> g{0.f, 1.f};
6     for (auto& x : noise_tbl) x = g(rng0);
7 }
8 size_t nidx = 0; // indice ciclico
9
10 // --- Bucle por muestra (hot path RT) ---
11 // sigma_n = sqrt(10^(noise_pdb/10)) [ec. eq:awgn_sigma]
12 const float nsigma = std::sqrt(std::pow(10.0f, noise_pdb / 10.0f));
13 for (ssize_t i = 0; i < n; ++i) {
14     // w[n] = sigma_n * (N_I + j*N_Q) [ec. eq:awgn]
15     // N_I, N_Q ~ N(0,1) independientes, tomados de la tabla ciclica
16     out_buf[i] += std::complex<float>{
17         nsigma * noise_tbl[nidx & (NOISE_TBL_SZ - 1u)], // componente I
18         nsigma * noise_tbl[(nidx + 1u) & (NOISE_TBL_SZ - 1u)] // componente Q
19     };
20     nidx += 2u;
21 }
```

Listing 9: Generación de ruido AWGN con tabla precalculada — `ChannelSimulator.cpp`, líneas 78–86 y 196–206

3.7 Tabla de parámetros del canal

Todos los campos pertenecen a `ChannelParams.hpp`.

Parámetro	Campo	Defecto	Descripción
<i>Parámetros de sistema</i>			
Potencia TX	tx_power_db	0 dB	Escala TX: $10^{P_{TX}/20}$
Ganancia RX	rx_gain_db	0 dB	Escala RX: $10^{G_{RX}/20}$
Ruido AWGN	noise_power_db	-60 dB	Piso ruido: $\sigma_n = 10^{N/20}$
Tipo antena	antenna_type	0 (Omni)	0 = omni, 1 = directiva Gauss.
Ancho haz	antenna_beamwidth_deg	30°	θ_{3dB} para directiva
N_T	num_targets	1	Núm. targets activos (0-3)
<i>Parámetros por target (×3)</i>			
R_i	distance	25 m	Distancia al target
v_i	velocity	10 m/s	Velocidad radial (+: acercándose)
θ_i	angle_deg	0°	Ángulo off-boresight (directiva)
A_m	micro_doppler_amp	0 rad	Amplitud modulación micro-Doppler
f_m	micro_doppler_freq	10 Hz	Frecuencia vibración micro-Doppler
σ_{RCS}	rcs	1 m ²	Sección eficaz radar del target

4 Canal transparente: ChannelSimulator_void

El ChannelSimulator_void implementa un canal completamente transparente: recibe cada frame I/Q por ZMQ PULL (puerto 5558) y lo retransmite **sin ninguna modificación** por ZMQ PUSH (puerto 5559):

$$y[n] = x[n] \quad \forall n \quad (16)$$

Su propósito es proporcionar una **referencia experimental** que permite:

- Verificar que el pipeline ZMQ completo funciona correctamente.
- Comprobar que, sin objetos en el canal, el mapa Rango-Doppler no muestra picos espurios significativos (solo la correlación cruzada TX/TX cerca del origen, que corresponde a la señal directa).
- Depurar el procesado radar aislando los efectos del canal.

```

1 // Canal transparente: recv() -> send() sin modificacion
2 // y[n] = x[n] [ec. eq:void_channel]
3 static void signal_loop(RadioInterface& radio, size_t buffer_size) {
4     std::vector<std::complex<float>> buf(buffer_size);
5     uint64_t frames = 0;
6
7     while (g_running.load(std::memory_order_relaxed)) {
8         const ssize_t n = radio.recv(buf.data(), buffer_size);
9         if (n <= 0) continue; // timeout o error - reintentar
10
11         radio.send(buf.data(), static_cast<size_t>(n)); // retransmitir intacto
12         ++frames;
13
14         // Indicador de actividad: imprimir cada 1000 frames
15         if (frames % 1000u == 0u)
16             std::printf("[VOID] Tramas retransmitidas: %llu\n",
17                         static_cast<unsigned long long>(frames));
18     }
19 }
```

Listing 10:
Bucle principal del canal void — channel_sim/src/ChannelSimulator_void.cpp, líneas 30-49

Resultado esperado con canal void

- Al ejecutar `plot_sensing_fast.py` con el canal void activo:
- El mapa Rango-Doppler no debe mostrar picos de alta amplitud fuera del bin (0,0) (correlación directa).
 - La presencia del bin (0,0) es normal y se debe a la correlación TX/TX de la señal piloto OFDM consigo misma.
 - Si aparecen picos en bins distintos de (0,0), puede indicar multipath en la interfaz loopback o un problema de configuración.

5 Cambios al código original de OpenISAC

Esta sección describe con detalle todos los cambios introducidos respecto al repositorio original de OpenISAC en GitHub. Los cambios se dividen en: (1) archivos nuevos, (2) archivos modificados, y (3) correcciones de bugs críticos detectados durante el desarrollo de la extensión ZMQ SIL.

5.1 Archivos nuevos

Archivo	Descripción
channel_sim/src/ChannelSimulator.cpp	Canal simulado completo. Implementa el modelo físico descrito en la Sección 3, pipeline ZMQ, TUI ncurses interactiva y control UDP en tiempo real.
channel_sim/src/ChannelSimulator_void.cpp	Canal transparente. Pasa la señal sin modificar (referencia nula).
channel_sim/include/ChannelParams.hpp	Parámetros del canal. Todos los campos son <code>std::atomic<></code> , permitiendo acceso concurrente seguro desde el hilo RT y la TUI/UDP.
channel_sim/src/ZmqInterface.cpp	Interfaz ZeroMQ PUSH/PULL para el simulador de canal. Hace BIND en los puertos 5558 (recepción) y 5559 (transmisión).
channel_sim/include/ZmqInterface.hpp	Cabecera de <code>ZmqInterface</code> y estructura <code>ZmqChannelSimConfig</code> .
scripts/run_modulator.bash	Watchdog: reinicia <code>OFDMModulator --zmq</code> automáticamente tras cualquier terminación no deseada. Parada limpia con Ctrl+C en el terminal del watchdog.

5.2 Archivos modificados

5.2.1 src/OFDMModulator.cpp

(a) **Flag --zmq en la CLI** El parser de argumentos de línea de comandos se extiende para reconocer el flag `--zmq` que activa el modo SIL:


```

1 bool zmq_mode = false;
2 // ...
3 for (int i = 1; i < argc; ++i) {
4     if (std::string(argv[i]) == "--zmq") {
5         zmq_mode = true;
6     }
7 }
8 cfg.zmq_mode = zmq_mode;
9 if (zmq_mode) {
10     // Usar YAML paired_frame_queue_size, con minimo 256
11     // (la latencia ZMQ ~10-30 frames requiere cola ampliada)
12     if (cfg.paired_frame_queue_size < 256) {
13         cfg.paired_frame_queue_size = 256;
14     }
15 }

```

Listing 11: Detección del flag -zmq — OFDMModulator.cpp

Nota sobre paired_frame_queue_size

En el código original, la cola de pareado TX/RX (PairedFrameQueue) tiene un tamaño de 8 frames. El modo ZMQ introduce una latencia variable de 10-30 frames entre TX y RX (latencia de la red loopback + procesado del ChannelSimulator). Con una cola de 8 frames, el modulador descartaría frames TX constantemente. El mínimo de 256 frames proporciona ≈ 590 ms de margen (a 434 Hz de frame rate), suficiente para absorber ráfagas de latencia.

(b) Inicialización de sockets ZMQ En modo --zmq, la función `_init_usrp()` omite toda la inicialización de hardware (USRP, streams) y en su lugar crea los sockets ZeroMQ:

```

1 #ifndef OPENISAC_ZMQ_SUPPORT
2 if (_cfg.zmq_mode) {
3     // Contexto ZMQ con 1 hilo I/O
4     _zmq_ctx = std::make_unique<zmq::context_t>(1);
5
6     // Socket TX (PUSH): envia frames al ChannelSimulator (puerto 5558)
7     // HWM=4: si el buffer del canal esta lleno, los sends bloquean 500ms y descartan
8     _zmq_push = std::make_unique<zmq::socket_t>(*_zmq_ctx, zmq::socket_type::push);
9     _zmq_push->set(zmq::sockopt::sndhwm, 4);
10    _zmq_push->set(zmq::sockopt::sndtimeo, 500); // 500ms timeout
11    _zmq_push->connect("tcp://127.0.0.1:5558");
12
13    // Socket RX sensing (PULL): recibe frames procesados (puerto 5559)
14    // rcvtimeo=100ms: timeout corto para no bloquear el hilo indefinidamente
15    _zmq_sensing_pull = std::make_unique<zmq::socket_t>(*_zmq_ctx, zmq::socket_type::pull);
16    _zmq_sensing_pull->set(zmq::sockopt::rcvtimeo, 100);
17    _zmq_sensing_pull->connect("tcp://127.0.0.1:5559");
18    return; // sin inicializacion de USRP
19 }
20 #endif
21 // ... inicializacion normal de USRP ...

```

Listing 12: Creación de sockets ZMQ en lugar de USRP — OFDMModulator.cpp

(c) Hilo `_zmq_sensing_rx_proc` Nuevo hilo dedicado a recibir los frames del canal (puerto 5559) e inyectarlos en el pipeline de sensing:

```

1 void _zmq_sensing_rx_proc() {
2     const size_t frame_size = _cfg.samples_per_frame();
3     AlignedVector frame_buf(frame_size);

```



```

4   size_t frame_received = 0;
5
6   while (_running.load(std::memory_order_relaxed)) {
7       zmq::message_t msg;
8       try {
9           auto res = _zmq_sensing_pull->recv(msg, zmq::recv_flags::none);
10          if (!res || msg.size() == 0) continue;
11      } catch (const zmq::error_t& e) {
12          if (e.num() == ETERM) break; // contexto destruido - salida limpia
13          continue;
14      }
15
16      // Ensamblar frames OFDM completos a partir de mensajes ZMQ de tamaño variable
17      const size_t msg_samps = msg.size() / sizeof(std::complex<float>);
18      const auto* src = static_cast<const std::complex<float>*>(msg.data());
19      size_t src_pos = 0;
20
21      while (src_pos < msg_samps) {
22          const size_t to_copy = std::min(frame_size - frame_received,
23                                         msg_samps - src_pos);
24          std::memcpy(frame_buf.data() + frame_received, src + src_pos,
25                    to_copy * sizeof(std::complex<float>));
26          frame_received += to_copy;
27          src_pos += to_copy;
28
29          if (frame_received >= frame_size) {
30              const uint64_t seq = _zmq_rx_frame_seq.fetch_add(1);
31              for (auto& ch : _sensing_channels) {
32                  // Adquirir buffer del pool (NO asignar heap nuevo por frame):
33                  // evita fuga de memoria de 14 GB en 37 s (ver Sección 6.3)
34                  AlignedVector ch_buf = ch->acquire_rx_frame();
35                  if (ch_buf.size() != frame_size) ch_buf.resize(frame_size);
36                  std::copy(frame_buf.begin(), frame_buf.end(), ch_buf.begin());
37                  ch->push_zmq_rx_frame(std::move(ch_buf), seq);
38                  // Si push falla (cola llena), push_zmq_rx_frame
39                  // libera el buffer al pool automáticamente.
40              }
41              frame_received = 0;
42          }
43      }
44  }
45  }

```

Listing 13: Hilo de recepción ZMQ sensing — OFDMModulator.cpp, líneas 1432-1482

(d) Hilo `_tx_proc` — modo ZMQ El hilo transmisor se adapta para enviar los frames via ZMQ en lugar de al stream USRP. Se añade una pausa temporal artificial para mantener la cadencia de frames:

```

1  // Envío del frame via ZMQ PUSH (en lugar de uhd::tx_stream::send())
2  const bool ok = _zmq_push->send(msg, zmq::send_flags::none).has_value();
3  // ...
4
5  // Guardia de underflow: solo activa el reinicio si NO estamos en modo ZMQ.
6  // En ZMQ, un fallo de send es un timeout de red (HWM), no un underflow HW.
7  if (!_cfg.zmq_mode && sent < frame_to_send.samples.size()) {
8      _tx_underflow_restart_requested.store(true);
9  }
10
11 // Pacing temporal: dormir el tiempo equivalente a un frame
12 // para no saturar el canal ZMQ ni generar latencia acumulada.
13 // T_frame = samples_per_frame / sample_rate [segundos]
14 if (_cfg.zmq_mode) {
15     std::this_thread::sleep_for(std::chrono::duration<double>(
16         static_cast<double>(_cfg.samples_per_frame()) / _cfg.sample_rate));
17 }

```

Listing 14: Envío ZMQ y pacing temporal — OFDMModulator.cpp

(e) Registro de SIGTERM El código original solo registraba el manejador de señal para SIGINT (Ctrl+C). SIGTERM (señal de terminación estándar de Linux/systemd) usaba el handler por defecto, que termina el proceso **sin ejecutar ningún código de limpieza** y sin imprimir mensajes de error:

```
1 // Código original (solo SIGINT):
2 std::signal(SIGINT, &signal_handler);
3
4 // Código corregido (SIGINT + SIGTERM):
5 std::signal(SIGINT, &signal_handler);
6 std::signal(SIGTERM, &signal_handler); // terminar limpiamente con kill/systemd
```

Listing 15: Registro de SIGTERM — OFDMModulator.cpp

5.2.2 src/SensingChannel.cpp / include/SensingChannel.hpp

(a) Método start_zmq_rx() Nuevo método público que arranca el hilo de procesamiento de sensing en modo ZMQ, sin inicializar el hardware RX de la USRP:

```
1 void SensingChannel::start_zmq_rx() {
2     _stop_requested.store(false);
3     _compute.bistatic_active = false;
4     _compute.next_delay_estimation_frame_seq = 0;
5     _rx_io.stream_start_time = uhd::time_spec_t(0.0);
6     _rx_io.next_rx_frame_seq = 0;
7     // Activar modo ZMQ loopback en la cola de pareado:
8     // pop_pair() usara timeout en lugar de bloqueo infinito
9     // para que el hilo de sensing no quede bloqueado si se descartan frames TX
10    _rx_io.paired_queue->set_zmq_loopback(true);
11    _compute.sensing_sender.start();
12    // Sin hilo _rx_io.rx_thread: los frames los provee zmq_sensing_rx_proc
13    _compute.sensing_thread = std::thread(&SensingChannel::_sensing_loop, this);
14 }
```

Listing 16: start_zmq_rx() — SensingChannel.cpp, líneas 891-903

(b) Métodos acquire_rx_frame() / release_rx_frame() Nuevos métodos para exponer el pool interno de buffers RX al hilo ZMQ externo, necesarios para resolver la fuga de memoria descrita en la Sección 5.3:

```
1 // Adquirir buffer preasignado del pool (sin asignacion de heap)
2 AlignedVector SensingChannel::acquire_rx_frame() {
3     return _rx_io.rx_frame_pool.acquire();
4 }
5
6 // Devolver buffer al pool (sin liberar memoria)
7 void SensingChannel::release_rx_frame(AlignedVector&& buf) {
8     _rx_io.rx_frame_pool.release(std::move(buf));
9 }
```

Listing 17: Nuevos métodos de pool — SensingChannel.cpp / SensingChannel.hpp

(c) PairedFrameQueue: modo ZMQ loopback La clase interna PairedFrameQueue se extiende con un mecanismo de **timeout en el lado TX** para el modo ZMQ. Sin este mecanismo, si la cola TX se llena y se descartan frames TX, el hilo de sensing se bloquea indefinidamente esperando el frame TX descartado (que nunca llegará):

```
1 // Nueva bandera atomica: activada por set_zmq_loopback(true)
2 std::atomic<bool> _zmq_loopback{false};
3 void set_zmq_loopback(bool v) { _zmq_loopback.store(v); }
```

```

4
5 // Variante con timeout de 200 ms para modo ZMQ
6 bool _pop_next_tx_zmq(TxSymbolsFrame& out, bool& timed_out) {
7     timed_out = false;
8     using Clock = std::chrono::steady_clock;
9     const auto deadline = Clock::now() + std::chrono::milliseconds(200);
10    SPSCBackoff backoff;
11    while (Clock::now() < deadline) {
12        if (_tx_q.try_pop(out)) return true;
13        if (_closed.load()) return false; // cola cerrada
14        backoff.pause();
15    }
16    timed_out = true;
17    return false; // timeout: frame TX descartado, resincronizan
18 }
19
20 // En pop_pair(): si hay timeout, descartar el frame RX y reintentar
21 if (_zmq_loopback.load()) {
22     bool timed_out = false;
23     if (!_pop_next_tx_zmq(tx_item, timed_out)) {
24         _recycle_one(std::move(rx_item.samples));
25         if (timed_out) { have_rx = false; continue; } // descartar RX, reiniciar
26         return false; // cola cerrada
27     }
28 }

```

Listing 18: Timeout ZMQ en PairedFrameQueue — SensingChannel.cpp, líneas 370-412

(d) push_zmq_rx_frame() — liberación al pool en caso de fallo Cuando la cola RX está llena y el push falla, el buffer se devuelve al pool (en lugar de descartarse):

```

1 bool SensingChannel::push_zmq_rx_frame(AlignedVector&& samples, uint64_t frame_seq) {
2     RxSymbolsFrame rx_item;
3     rx_item.samples = std::move(samples);
4     rx_item.frame_seq = frame_seq;
5     if (!_rx_io.paired_queue->push_rx(std::move(rx_item))) {
6         LOG_RT_WARN_HZ(5) << "[Sensing CH " << _rx_io.logical_id
7             << "] ZMQ RX queue full, dropping frame " << frame_seq;
8         // SPSCRingBuffer::try_push NO mueve el valor si la cola esta llena,
9         // por lo que rx_item.samples es aun valido aqui.
10        // Devolver el buffer al pool para mantener el ciclo acquire/release balanceado.
11        _rx_io.rx_frame_pool.release(std::move(rx_item.samples));
12        return false;
13    }
14    _rx_io.next_rx_frame_seq = frame_seq + 1;
15    return true;
16 }

```

Listing 19: push_zmq_rx_frame con liberación al pool — SensingChannel.cpp, líneas 905-916

5.2.3 CMakeLists.txt

Se añaden dos nuevos targets de compilación condicionales al bloque `if (CHANNEL_SIM_ZMQ_FOUND)`:

Listing 20: Nuevos targets en CMakeLists.txt

```

# Target ChannelSimulator (canal fisico con TUI)
add_executable(ChannelSimulator
    channel_sim/src/ChannelSimulator.cpp
    channel_sim/src/ZmqInterface.cpp
    src/AsyncLogger.cpp)
target_link_libraries(ChannelSimulator PRIVATE

```

```

    ${ZMQ_LIBRARIES} ncurses yaml-cpp uhd fftw3f ...)

# Target ChannelSimulator_void (canal transparente, sin ncurses)
add_executable(ChannelSimulator_void
    channel_sim/src/ChannelSimulator_void.cpp
    channel_sim/src/ZmqInterface.cpp
    src/AsyncLogger.cpp)
target_link_libraries(ChannelSimulator_void PRIVATE
    ${ZMQ_LIBRARIES} yaml-cpp uhd ...)

```

La variable de compilación `OPENISAC_ZMQ_SUPPORT` se define cuando ZMQ está disponible y se propaga a `OFDMModulator` y `OFDMDemodulator`, activando todas las secciones `#ifdef OPENISAC_ZMQ_SUPPORT`.

5.3 Correcciones de bugs críticos detectados

Durante el desarrollo de la extensión ZMQ SIL se identificaron y corrigieron tres bugs críticos presentes en el código base original:

5.3.1 Bug 1: Bloqueo infinito en PairedFrameQueue (modo ZMQ)

Descripción: En modo ZMQ, cuando la cola TX (`PairedFrameQueue`) se llenaba y los frames TX se descartaban, el hilo de sensing llamaba a `_pop_next_tx()`, que gira indefinidamente esperando el siguiente frame TX. El frame descartado nunca llegaría, bloqueando el hilo para siempre.

Causa raíz: `_pop_next_tx()` no tiene mecanismo de timeout; fue diseñado para uso con hardware donde los frames TX nunca se descartan.

Solución: Añadir `_pop_next_tx_zmq()` con timeout de 200 ms y activarlo con el flag `set_zmq_loopback(true)` en `start_zmq_rx()`. Véase el código en la Sección anterior.

5.3.2 Bug 2: Fuga de memoria de 14,7 GB en _zmq_sensing_rx_proc

Descripción: El proceso `OFDMModulator --zmq` terminaba involuntariamente ≈ 37 segundos después del arranque, eliminado por el OOM killer del sistema operativo.

Causa raíz: El hilo `_zmq_sensing_rx_proc` asignaba un nuevo `AlignedVector` de ≈ 921 KB en cada frame recibido. El hilo de sensing lo procesaba y lo **liberaba al pool** mediante `rx_frame_pool.release()`. Sin embargo, el hilo ZMQ nunca adquiría del pool: solo liberaba. El pool crecía sin límite:

$$16.000 \text{ frames} \times 921 \text{ KB/frame} \approx 14,7 \text{ GB} \quad (17)$$

A ≈ 434 Hz de frame rate, la acumulación alcanza el límite de RAM en ≈ 37 segundos (con 16 GB de RAM instalada).

Solución: El hilo ZMQ ahora llama a `ch->acquire_rx_frame()` para obtener buffers preasignados del pool. El hilo de sensing los devuelve con `rx_frame_pool.release()`. El pool permanece en estado de equilibrio estacionario.

5.3.3 Bug 3: SIGTERM sin manejador registrado

Descripción: El proceso OFDMModulator terminaba silenciosamente (sin mensajes de error) cuando recibía una señal SIGTERM (p.ej. de kill, systemd o taskset).

Causa raíz: El código original solo registraba el handler de señal para SIGINT (Ctrl+C). SIGTERM usaba la acción por defecto del kernel, que termina el proceso inmediatamente sin ejecutar destructores ni código de limpieza.

Solución: Añadir `std::signal(SIGTERM, &signal_handler)` junto a la ya existente línea de SIGINT.

6 Manual de instalación

6.1 Dependencias del sistema

Listing 21: Instalación de dependencias en Ubuntu 22.04+

```
# Dependencias base de OpenISAC (igual que el repositorio original)
sudo apt update
sudo apt install -y \
    cmake build-essential pkg-config \
    libuhd-dev uhd-host \
    libboost-all-dev \
    libfftw3-dev \
    libyaml-cpp-dev \
    libncurses-dev

# ZeroMQ (necesario para el modo ZMQ SIL)
sudo apt install -y libzmq3-dev libczmq-dev

# cppzmq (cabeceras C++ de ZMQ – si no está en el sistema)
sudo apt install -y cppzmq-dev
# Alternativa: descargar zmq.hpp de https://github.com/zeromq/cppzmq

# AFF3CT (codec LDPC) – compilar desde fuentes
git clone --recursive https://github.com/aff3ct/aff3ct.git
cd aff3ct && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release \
    -DAFF3CT_COMPILE_SHARED_LIB=ON \
    -DAFF3CT_COMPILE_STATIC_LIB=OFF
make -j$(nproc)
sudo make install && sudo ldconfig
cd ../../
```

6.2 Compilación

Listing 22: Compilación del proyecto

```
cd fork_OpenISAC

# Configurar (Release con optimizaciones O2)
cmake -B build -DCMAKE_BUILD_TYPE=Release

# Compilar todos los targets
cmake --build build -j$(nproc)

# Binarios generados en build/:
# OFDMModulator – transmisor/receptor ISAC
```

```
# OFDMDemodulator - demodulador (modo bistico / UE)
# ChannelSimulator - canal simulado con TUI (requiere ZMQ)
# ChannelSimulator_void - canal transparente (requiere ZMQ)
```

Si ZMQ no está disponible en el sistema, cmake mostrará un aviso y omitirá los targets de canal simulado:

```
-- ZeroMQ (libzmq + zmq.hpp) not found - ChannelSimulator will NOT be built.
```

6.3 Configuración YAML

Copiar una de las plantillas de configuración al directorio build/:

```
cp config/Modulator_B210.yaml build/Modulator.yaml
# Verificar los parametros criticos:
# sample_rate: 50000000 (Hz) - frecuencia de muestreo
# center_freq: 2400000000 (Hz) - frecuencia portadora
# fft_size: 1024 - numero de subportadoras
# num_symbols: 100 - simbolos OFDM por frame
# cp_length: 128 - longitud del prefijo ciclico
```

Para el modo ZMQ SIL, el bloque channel_sim del YAML debe estar **descommentado** (controla los puertos ZMQ y el puerto de control UDP):

Listing 23: Bloque channel_sim en Modulator.yaml

```
channel_sim:
  control_port: 9998      # Puerto UDP para comandos de control remoto
  buffer_size: 4096      # Tamano de buffer ZMQ (muestras)
  zmq:
    rx_bind_addr: "tcp://*:5558" # ChannelSim recibe TX en este puerto
    tx_bind_addr: "tcp://*:5559" # ChannelSim envia RX en este puerto
```

7 Manual de uso paso a paso

7.1 Modo 1: Hardware real (USRP)

Este es el modo de operación original de OpenISAC. Requiere hardware USRP.

Prerrequisito hardware

Conectar el USRP al PC via USB3 (B210) o SFP/PCIe (X310). Verificar la conexión: `uhd_find_devices`. El USRP debe aparecer en la salida.

7.1.1 Terminal 1 — Configuración del sistema (una vez por sesión)

```
# Governor de CPU a performance (reduce jitter de frecuencia)
sudo ./scripts/set_performance.bash

# Aislar 6 cores para la aplicacion (OS usa solo cores 6-7)
# Sustituir 0-5 por el rango adecuado para el procesador del sistema
sudo ./scripts/isolate_cpus.bash 0-5
```

7.1.2 Terminal 2 — OFDMModulator (sin flag -zmq)

```
cd build
# Lanzar con afinidad de CPU y prioridad RT:
sudo ../scripts/isolate_cpus.bash run ./OFDMModulator Modulator.yaml
```

El modulador se conecta al USRP, sintoniza la frecuencia y comienza a transmitir frames OFDM. Los datos de sensing se envían por UDP al puerto 8888.

7.1.3 Terminal 3 — Visualización

```
cd build
python3 ../scripts/plot_sensing_fast.py
```

Se abre la ventana de visualización con el mapa Rango-Doppler en tiempo real. En ausencia de objetos, solo debería aparecer el bin DC (señal directa).

7.2 Modo 2: Canal transparente (ChannelSimulator_void)

Este modo ejecuta la cadena ISAC completa sin hardware USRP y sin efectos de canal. Es útil para verificar el funcionamiento del pipeline y establecer una referencia nula.

Orden de arranque obligatorio

El ChannelSimulator (o _void) debe arrancar **antes** que el OFDMModulator. El simulador hace `bind()` en los puertos ZMQ; el modulador hace `connect()`. Si se invierten, el modulador no encontrará el servidor ZMQ.

7.2.1 Terminal 1 — ChannelSimulator_void

```
cd build
./ChannelSimulator_void Modulator.yaml
```

Salida esperada:

```
[VOID] Canal transparente iniciado
[VOID] RX (PULL): tcp://*:5558
[VOID] TX (PUSH): tcp://*:5559
[VOID] Buffer: 115200 muestras
[VOID] Ctrl+C para salir.

[VOID] Tramas retransmitidas: 1000
[VOID] Tramas retransmitidas: 2000
...
```

7.2.2 Terminal 2 — OFDMModulator en modo ZMQ

```
cd build
./OFDMModulator --zmq Modulator.yaml
```

7.2.3 Terminal 3 — Visualización

```
cd build
python3 ../scripts/plot_sensing_fast.py
```

Resultado esperado — canal void

El mapa Rango-Doppler **no debe mostrar picos de alta amplitud** fuera del bin (rango=0, Doppler=0). Este resultado confirma que:

1. El pipeline ZMQ funciona correctamente de extremo a extremo.
2. No existen artefactos introducidos por el software de simulación.
3. Cualquier pico que aparezca en los experimentos con canal simulado es atribuible exclusivamente al modelo de canal.

7.2.4 Parada limpia

1. Ctrl+C en Terminal 2 (OFDMModulator).
2. Ctrl+C en Terminal 1 (ChannelSimulator_void).

7.3 Modo 3: Canal simulado con efectos físicos (ChannelSimulator)

Este modo simula un canal radar monostático completo con los efectos físicos descritos en la Sección 3. Los parámetros se controlan en tiempo real mediante la TUI ncurses.

7.3.1 Terminal 1 — ChannelSimulator (con TUI)

```
cd build
./ChannelSimulator Modulator.yaml
```

Se abre la interfaz TUI ncurses. Para prioridad RT (hilo de señal en SCHED_FIFO/80):

```
sudo ./ChannelSimulator Modulator.yaml
```

7.3.2 Terminal 2 — OFDMModulator en modo ZMQ

```
cd build
./OFDMModulator --zmq Modulator.yaml
# Para watchdog automatico (recomendado para sesiones largas):
sudo ../scripts/run_modulator.bash
```

7.3.3 Terminal 3 — Visualización

```
cd build
python3 ../scripts/plot_sensing_fast.py
```

7.3.4 Uso de la TUI ncurses

CHANNEL SIMULATOR – PANEL DE CONTROL		SR:50MHz	Fc:2.400GHz
Parametro	Valor	Paso	
SISTEMA			
Potencia TX	0.000 dB	1	
Ganancia RX	0.000 dB	1	
Ruido AWGN	-60.000 dB	5	
Tipo de antena	Omnidireccional		
Targets	1		
TARGET 1			
> Distancia	25.000 m	10	
Velocidad	10.000 m/s	5	
MicroDop. Amp	0.000 rad	0.01	
MicroDop. Freq	10.000 Hz	1	
RCS	1.000 m2	0.5	
up/dn Navegar +/- <- -> Valor j/k Paso x0.5/x2 Q Salir			

Tecla	Acción
↑ / ↓	Navegar entre parámetros
→ / + / =	Incrementar valor en un paso
← / -	Decrementar valor en un paso
j	Paso ÷2 (ajuste más fino)
k	Paso ×2 (ajuste más grueso)
Q	Salir limpiamente del ChannelSimulator

7.3.5 Experimentos sugeridos

Experimento 1 — Target estático (verificación de rango):

1. Configurar: Targets=1, Distancia=100 m, Velocidad=0 m/s, RCS=1 m².
2. En el mapa Rango-Doppler, debe aparecer un pico en el bin de rango correspondiente a $d = \lfloor 2 \times 100/c \times f_s \rfloor$ y en el bin Doppler=0.
3. Variar la distancia con las flechas: el pico debe desplazarse en el eje de rango.

Experimento 2 — Target con velocidad (verificación Doppler):

1. Configurar: Targets=1, Distancia=50 m, Velocidad=20 m/s, RCS=5 m².
2. El pico debe aparecer desplazado en el eje Doppler: $f_d = 2 \times 20 \times f_c/c$ Hz.
3. Cambiar el signo de la velocidad: el pico debe saltar al lado opuesto.

Experimento 3 — Micro-Doppler (verificación de firma):

1. Configurar: Targets=1, Dist=50 m, Vel=15 m/s, MicroDop.Amp=0.3 rad, MicroDop.Freq=25 Hz.
2. En la dimensión Doppler, deben aparecer *sidebands* simétricas a $\pm k \times 25$ Hz del Doppler principal.

3. El número y amplitud de sidebands depende de la amplitud A_m (expansión en series de Bessel de la modulación PM).

Experimento 4 — Antena directiva (verificación de beam pattern):

1. Configurar: Tipo antena = Directiva, Ancho haz = 20° , Target1: Dist=50 m, Vel=20 m/s, Ángulo= 0° (on-boresight).
2. Registrar la amplitud del pico.
3. Aumentar el ángulo a 10° : la amplitud debe caer a ≈ -3 dB.
4. Aumentar a 20° : la amplitud debe caer a ≈ -12 dB.

Experimento 5 — Múltiples targets:

1. Configurar: Targets=3, Target1: 50 m / 20 m/s, Target2: 150 m / -10 m/s, Target3: 300 m / 5 m/s.
2. Deben aparecer tres picos distintos en el mapa Rango-Doppler, cada uno en la celda (rango, Doppler) correspondiente.

Experimento 6 — Degradación por ruido:

1. Configurar un target visible. Subir *Ruido* AWGN desde -60 dB hacia 0 dB.
2. El pico del target debe ir desapareciendo progresivamente en el suelo de ruido.
3. La SNR en el mapa Rango-Doppler: $SNR_{RD} \approx A_{eco}^2 / (2\sigma_n^2) \times N_r \times N_d$ donde N_r y N_d son los tamaños de las FFT de rango y Doppler.

7.4 Control remoto por UDP (opcional)

El ChannelSimulator acepta comandos en el puerto control_port (9998 por defecto) para automatizar cambios de parámetros desde scripts:

Listing 24: Ejemplos de control UDP desde Python

```
python3 - <<'EOF'
import socket, struct

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Cambiar velocidad del Target 1 a 30.5 m/s (valor = vel * 100 = 3050)
s.sendto(b'VEL ' + struct.pack('>i', 3050), ('127.0.0.1', 9998))

# Cambiar distancia a 200.0 m (valor = dist * 10 = 2000)
s.sendto(b'DIST' + struct.pack('>i', 2000), ('127.0.0.1', 9998))

# Activar micro-Doppler: amp=0.3 rad (valor = amp * 10000 = 3000)
s.sendto(b'MDAF' + struct.pack('>i', 3000), ('127.0.0.1', 9998))

# Frecuencia micro-Doppler: 25 Hz (valor = freq * 100 = 2500)
s.sendto(b'MDFF' + struct.pack('>i', 2500), ('127.0.0.1', 9998))
EOF
```

8 Rendimiento y optimizaciones en tiempo real

8.1 Tabla de ruido precalculada

El generador estándar `std::normal_distribution` con `mt19937` tiene un coste de ≈ 15 ns/muestra. Para un frame de $N_{\text{sym}} = 100$ símbolos OFDM con $N_{\text{FFT}} + N_{\text{CP}} = 1024 + 128 = 1152$ muestras/símbolo, el frame tiene $115\,200$ muestras $\times 2$ (I/Q) = $230\,400$ valores Gaussianos:

$$T_{\text{RNG}} \approx 230\,400 \times 15 \text{ ns} = 3,5 \text{ ms}$$

El periodo de frame a $f_s = 50$ MHz es:

$$T_{\text{frame}} = 115\,200 / 50 \times 10^6 = 2,3 \text{ ms}$$

Por tanto, el generador estándar consumiría $> 100\%$ del presupuesto de tiempo de un frame, haciendo imposible la ejecución en tiempo real.

La tabla precalculada de 2^{17} muestras Gaussianas (512 KB) permite accesos a ≈ 2 ns/muestra (caché L3), reduciendo $T_{\text{RNG}} < 0,5$ ms, equivalente al $\approx 20\%$ del presupuesto de frame.

8.2 Fasor Doppler incremental

La multiplicación compleja de rotación es ≈ 4 operaciones FP (2 mul + 2 add), con latencia ≈ 4 ns en hardware x86 moderno. La alternativa (`cos/sin` por muestra) tiene latencia $\approx 10 - 15$ ns. El ahorro total por frame (3 targets):

$$(15 - 4) \text{ ns} \times 115\,200 \text{ muestras} \times 3 \text{ targets} \approx 3,8 \text{ ms}$$

8.3 Hilo de señal con prioridad RT

El hilo `signal_loop` (canal de señal) se ejecuta con política `SCHED_FIFO` y prioridad 80 cuando se lanza con `sudo`, garantizando que no sea desalojado por procesos del sistema operativo durante el procesado de un frame.

9 Resolución de problemas

Síntoma	Causa / Solución
Cannot load config: Modulator.yaml	El binario busca el YAML en el directorio de trabajo. Ejecutar desde build/ o pasar la ruta explícita: <code>./OFDMModulator --zmq ../build/Modulator.yaml</code>
El modulador termina ≈ 37 s después de arrancar	Bug de fuga de memoria corregido en esta extensión. Recompilar desde el repositorio actualizado.
Seq mismatch / drop de frames al inicio	El ChannelSimulator no ha arrancado antes que el modulador. Reiniciar en el orden correcto.
Mapa Rango-Doppler sin pico de target	Verificar: <code>num_targets ≥ 1</code> en la TUI, <code>tx_power_db</code> y <code>rsc > 0</code> , <code>noise_power_db ≤ -40 dB</code> .
TUI con caracteres extraños	Verificar locale UTF-8: <code>echo \$LANG</code> (debe mostrar <code>es_ES.UTF-8</code> o <code>en_US.UTF-8</code>).
[WARN] Sin prioridad RT	Lanzar ChannelSimulator con <code>sudo</code> para activar <code>SCHED_FIFO/80</code> en el hilo de señal.
ZMQ error: ETERM	El contexto ZMQ se ha destruido (normal en apagado). No es un error: indica salida limpia del hilo de recepción.
OFDMModulator --zmq termina nada más arrancar	El ChannelSimulator no ha arrancado aún. El socket TX (PUSH + HWM=4 + sndtimeo=500 ms) descarta frames si no hay receptor. Arrancar el canal primero.

10 Resumen de todos los cambios frente al repositorio original

Archivo	Tipo	Cambio
channel_sim/src/ChannelSimulator.cpp	Nuevo	Canal físico: retardo, Doppler, micro-Doppler, ec. radar, AWGN, TUI
channel_sim/src/ChannelSimulator_void.cpp	Nuevo	Canal transparente (referencia nula)
channel_sim/include/ChannelParams.hpp	Nuevo	Parámetros del canal (atómicos, thread-safe)
channel_sim/src/ZmqInterface.cpp	Nuevo	Interfaz ZMQ PUSH/PULL (BIND en 5558/5559)
channel_sim/include/ZmqInterface.hpp	Nuevo	Cabecera de ZmqInterface y ZmqChannelSimConfig
scripts/run_modulator.bash	Nuevo	Watchdog: reinicio automático del modulador
src/OFDMModulator.cpp	Modificado	(1) Flag - - zmq en CLI; (2) Inicialización ZMQ en lugar de USRP; (3) Hilo _zmq_sensing_rx_proc; (4) Pacing temporal TX; (5) Guard de underflow modo ZMQ; (6) SIGTERM registrado; (7) Fix fuga de memoria: acquire_rx_frame()
src/SensingChannel.cpp	Modificado	(1) Método start_zmq_rx(); (2) Método push_zmq_rx_frame(); (3) acquire_rx_frame() / release_rx_frame(); (4) PairedFrameQueue: timeout ZMQ loopback; (5) Fix liberación al pool en push fallido
include/SensingChannel.hpp	Modificado	Declaraciones de start_zmq_rx(), push_zmq_rx_frame(), acquire_rx_frame(), release_rx_frame()
CMakeLists.txt	Modificado	Targets ChannelSimulator y ChannelSimulator_void; define OPENISAC_ZMQ_SUPPORT
scripts/*.bash	Modificado	Conversión CRLF → LF (scripts inejecutables en Linux original)

OpenISAC ZMQ SIL Extension — Basado en el proyecto OpenISAC original (GitHub).

Documento de referencia para publicación en congreso ISAC.

Generado con XeLaTeX · fork_OpenISAC · Mayo 2026.